

RL with Sequences & LLMs

CS 185/285

Instructor: Sergey Levine
UC Berkeley

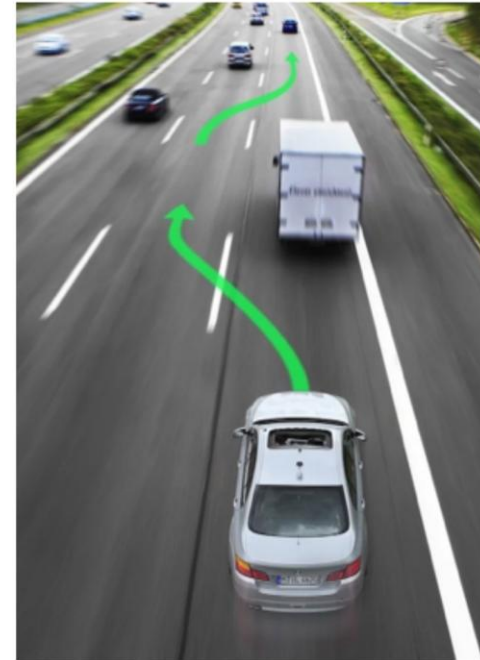
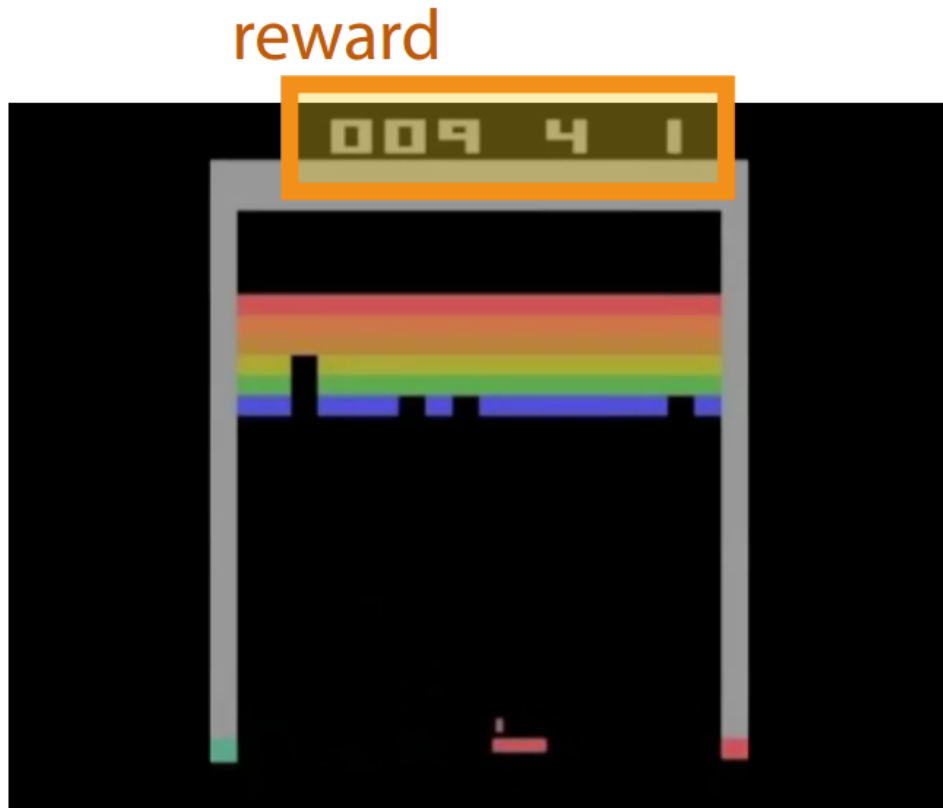


Part 1:

Reward learning & inverse RL

Why should we worry about learning rewards?

The reinforcement learning perspective



what is the reward?

Learning the optimality variable

$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t, \psi) \propto \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t))$$

$\uparrow$

reward parameters

given:

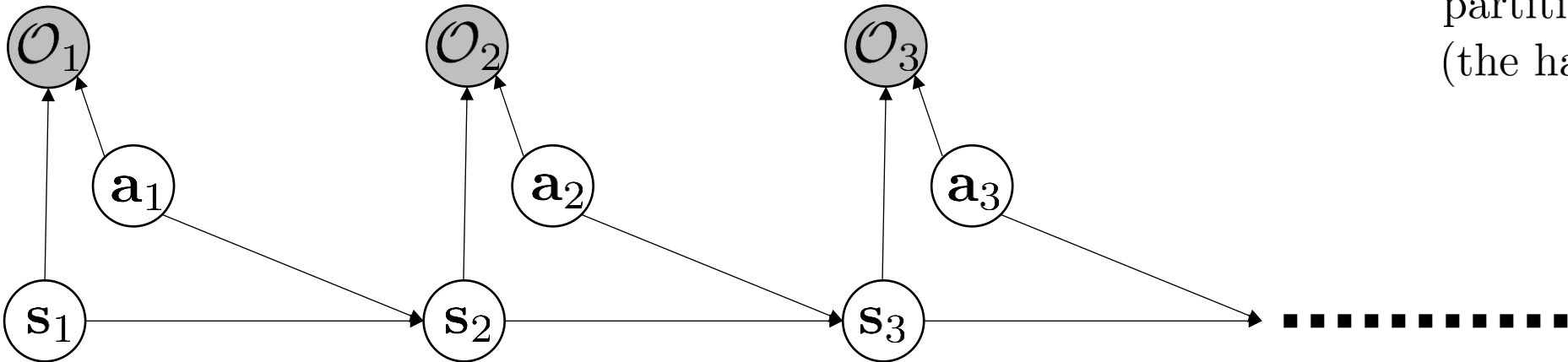
samples $\{\tau_i\}$ sampled from $\pi^*(\tau)$

$$p(\tau | \mathcal{O}_{1:T}, \psi) \propto \cancel{p(\tau)} \exp\left(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t)\right)$$

can ignore (independent of ψ)

maximum likelihood learning:
$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N \log p(\tau_i | \mathcal{O}_{1:T}, \psi) = \max_{\psi} \frac{1}{N} \sum_{i=1}^N r_\psi(\tau_i) - \log Z$$

$\nwarrow$
partition function
(the hard part)



The IRL partition function

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N r_{\psi}(\tau_i) - \log Z$$

$$Z = \int p(\tau) \exp(r_{\psi}(\tau)) d\tau$$

$$\nabla_{\psi} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \underbrace{\frac{1}{Z} \int p(\tau) \exp(r_{\psi}(\tau)) \nabla_{\psi} r_{\psi}(\tau) d\tau}_{p(\tau | \mathcal{O}_{1:T}, \psi)}$$

$$\nabla_{\psi} \mathcal{L} = E_{\tau \sim \pi^*(\tau)} [\nabla_{\psi} r_{\psi}(\tau_i)] - E_{\tau \sim p(\tau | \mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)]$$



estimate with expert samples



soft optimal policy under current reward

Estimating the gradient

$$\nabla_{\psi} \mathcal{L} = E_{\tau \sim \pi^*(\tau)} [\nabla_{\psi} r_{\psi}(\tau_i)] - E_{\tau \sim p(\tau | \mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)]$$



estimate with expert samples



soft optimal policy under current reward

idea: learn $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}, \psi)$ using any max-ent RL algorithm

$$J(\theta) = \sum_t E_{\pi(\mathbf{s}_t, \mathbf{a}_t)} [r_{\psi}(\mathbf{s}_t, \mathbf{a}_t)] + E_{\pi(\mathbf{s}_t)} [\mathcal{H}(\pi(\mathbf{a} | \mathbf{s}_t))]$$

then run this policy to sample $\{\tau_j\}$

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{M} \sum_{j=1}^M \nabla_{\psi} r_{\psi}(\tau_j)$$



sum over expert samples



sum over policy samples

More efficient sample-based updates

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{M} \sum_{j=1}^M \nabla_{\psi} r_{\psi}(\tau_j)$$


sum over expert samples

sum over policy samples

improve ~~learn~~ $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}, \psi)$ using any max-ent RL algorithm
(a little)
then run this policy to sample $\{\tau_j\}$

looks expensive! what if we use “lazy” policy optimization?

problem: estimator is now biased! wrong distribution!

solution 1: use importance sampling

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_{\psi} r_{\psi}(\tau_j) \quad w_j = \frac{p(\tau) \exp(r_{\psi}(\tau_j))}{\pi(\tau_j)}$$

Importance sampling

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_{\psi} r_{\psi}(\tau_j)$$

$$w_j = \frac{p(\tau) \exp(r_{\psi}(\tau_j))}{\pi(\tau_j)}$$


$$\frac{\cancel{p(\mathbf{s}_1)} \prod_t \cancel{p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)} \exp(r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{\cancel{p(\mathbf{s}_1)} \prod_t \cancel{p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)} \pi(\mathbf{a}_t|\mathbf{s}_t)}$$
$$= \frac{\exp(\sum_t r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{\prod_t \pi(\mathbf{a}_t|\mathbf{s}_t)}$$

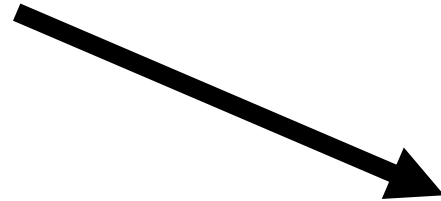
each policy update w.r.t. r_{ψ} brings us closer to the target distribution!

It looks a bit like a game...

initial
policy π



samples from $\pi_\theta(\tau)$



$$\nabla_\theta \mathcal{L} \approx \frac{1}{M} \sum_{j=1}^M \nabla_\theta \log \pi_\theta(\tau_j) r_\psi(\tau_j)$$

policy changed to make it *harder* to distinguish from demos

 human
demonstrations



samples from $\pi^*(\tau)$



$$\nabla_\psi \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_\psi r_\psi(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_\psi r_\psi(\tau_j)$$

demos are made more likely, samples less likely

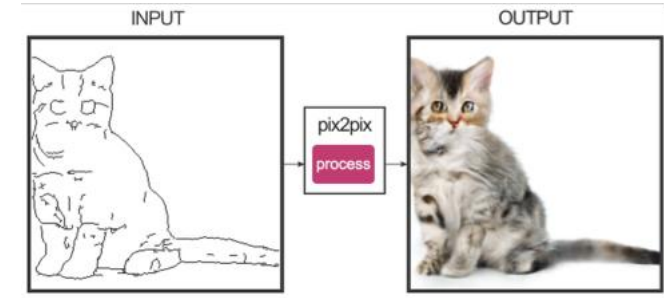
Generative Adversarial Networks



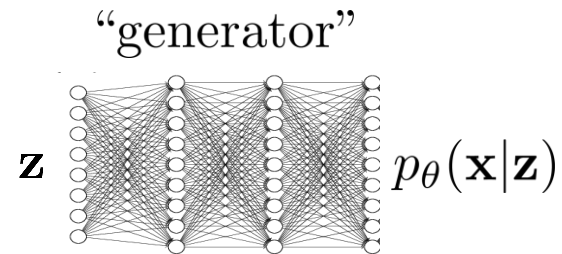
Zhu et al. '17



Arjovsky et al. '17



Isola et al. '17

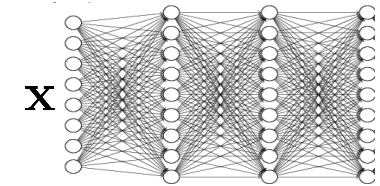


data ("demonstrations")



samples from $p^*(\mathbf{x})$

$$D(\mathbf{x}) = p_\psi(\text{real image}|\mathbf{x})$$

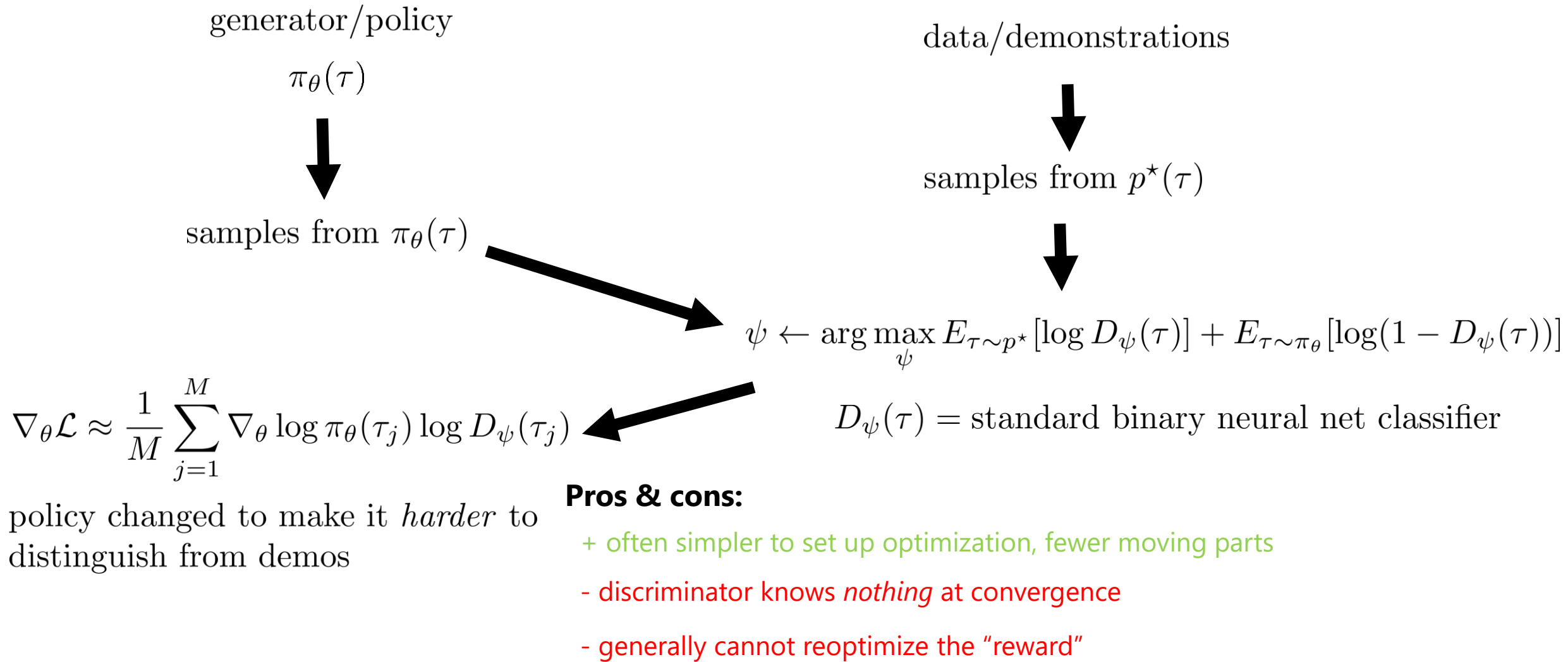


samples from $p_\theta(\mathbf{x})$

$$\psi = \arg \max_{\psi} \frac{1}{N} \sum_{\mathbf{x} \sim p^*} \log D_\psi(\mathbf{x}) + \frac{1}{M} \sum_{\mathbf{x} \sim p_\theta} \log(1 - D_\psi(\mathbf{x}))$$

$$\theta \leftarrow \arg \max_{\theta} E_{\mathbf{x} \sim p_\theta} \log D_\psi(\mathbf{x})$$

Can we just use a regular discriminator?



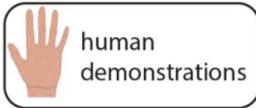
IRL as adversarial optimization

minimized maximized

reward function



robot attempt



human demonstrations

learns distribution $p(\tau)$ such that
demos have max likelihood
 $p(\tau) \propto \exp(r(\tau))$ (MaxEnt model)

$$D(\tau) = \frac{\frac{1}{Z} \exp(r(\tau))}{\frac{1}{Z} \exp(r(\tau)) + \pi(\tau)}$$

actually the
same thing!

False True

classifier



robot attempt

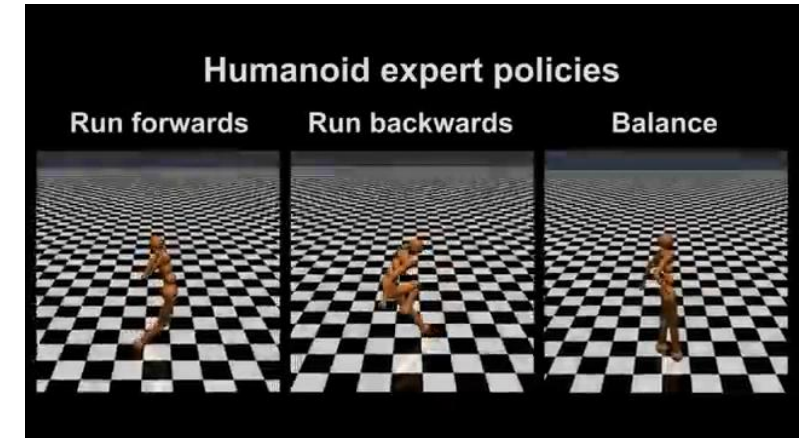


human demonstrations

$D(\tau)$ = probability τ is a demo

use $\log D(\tau)$ as “reward”

$D(\tau)$ = some classifier



Hausman, Chebotar, Schaal, Sukhatme, Lim

Motion Imitation



Reference Motion
(Mocap)



Learned Policy
(Simulation)

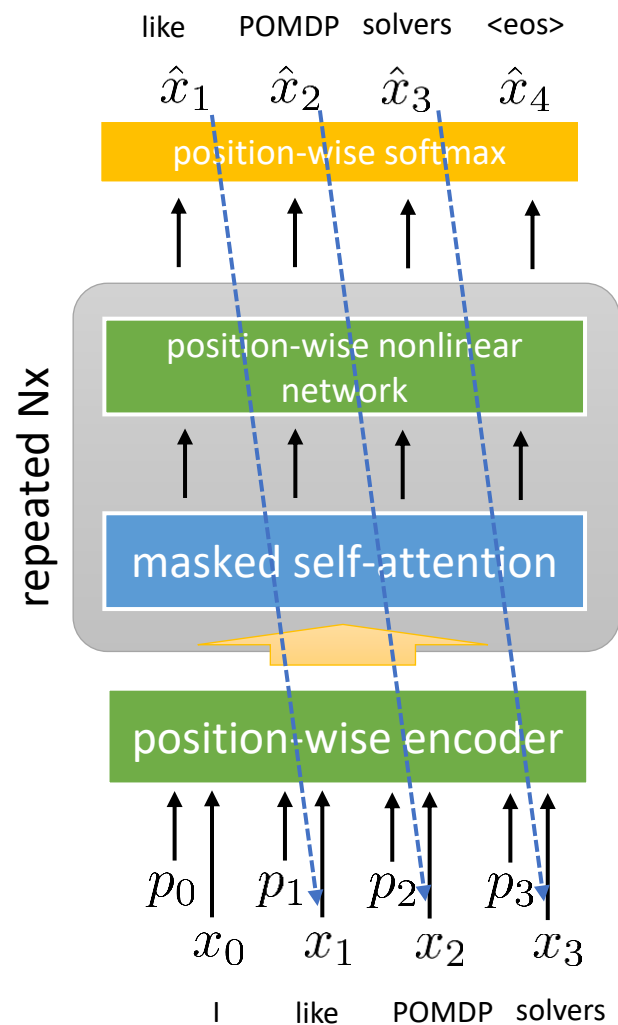
the goal is to train a simulated character to imitate the motion.

Peng, Kanazawa, Toyer, Abbeel, Levine

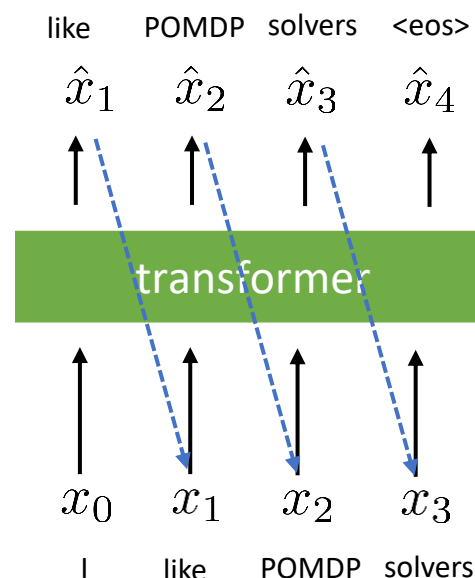
Part 2:

RL and LLMs: the basics

Language models



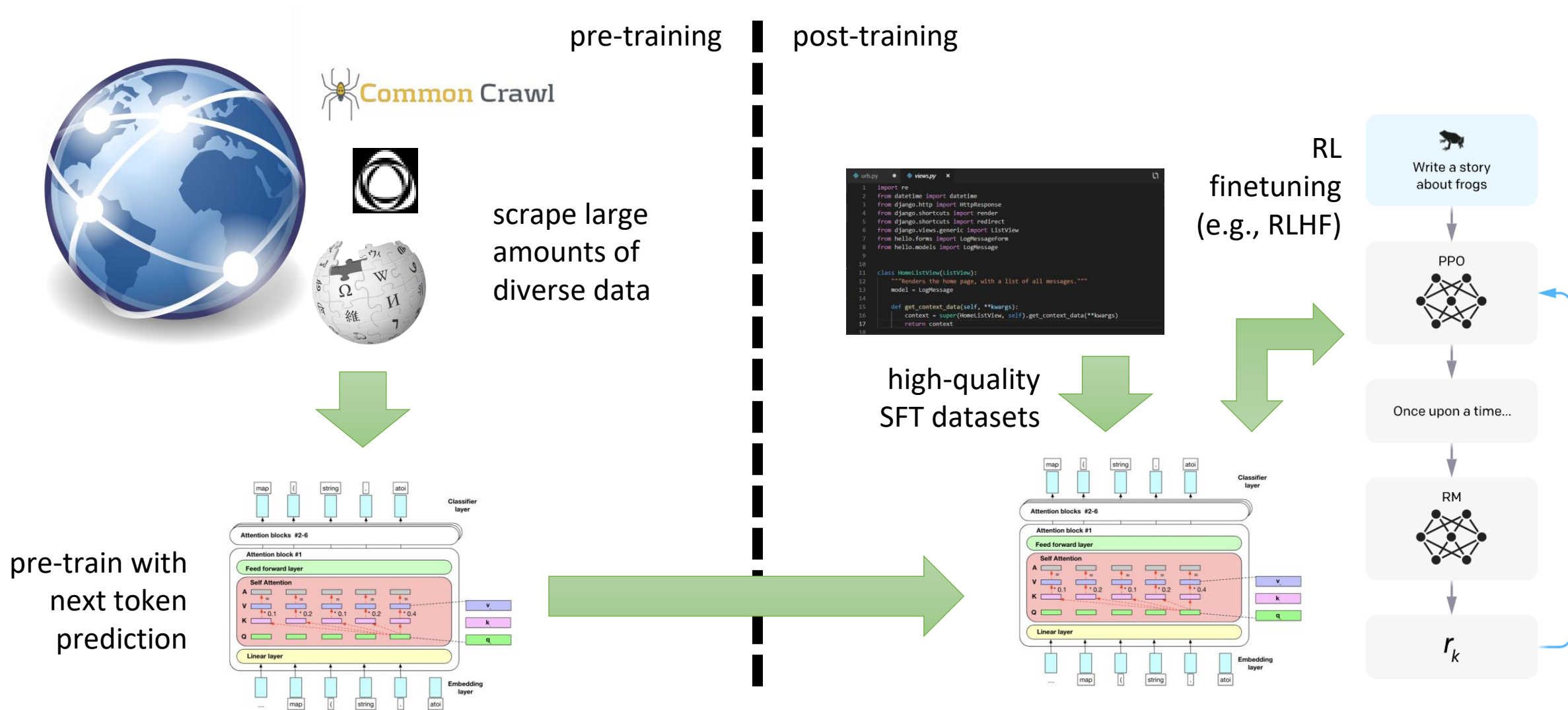
let's
simplify



- Language models are often trained with supervised learning
- But we can also train them with RL if what we want is to maximize some reward function, rather than just represent the data distribution
 - Why?
- Some questions:
 - What is the (PO)MDP?
 - What is the reward?
 - What algorithm to use?

We have a few choices to make!

The anatomy of LLM training



LLM post-training

pre-trained LLMs are **not** agents, assistants, or anything of the sort
they are “text completion” engines

...but completing *any* text requires significant “world knowledge”

Pat watches a demonstration of a bowling ball and a leaf being dropped at the same time in a vacuum chamber. Pat, who is a physicist, predicts that the bowling ball and the leaf will fall at the same rate.

Changing the last sentence of the prompt, we get:

...Pat, who has never seen this demonstration before, predicts that the bowling ball will fall to the ground first. This is incorrect. In a vacuum chamber, there is no air

Andreas et al., 2022

pre-training is what gives
the LLM *knowledge*

post-training is what tells
the LLM how to *use* this
knowledge

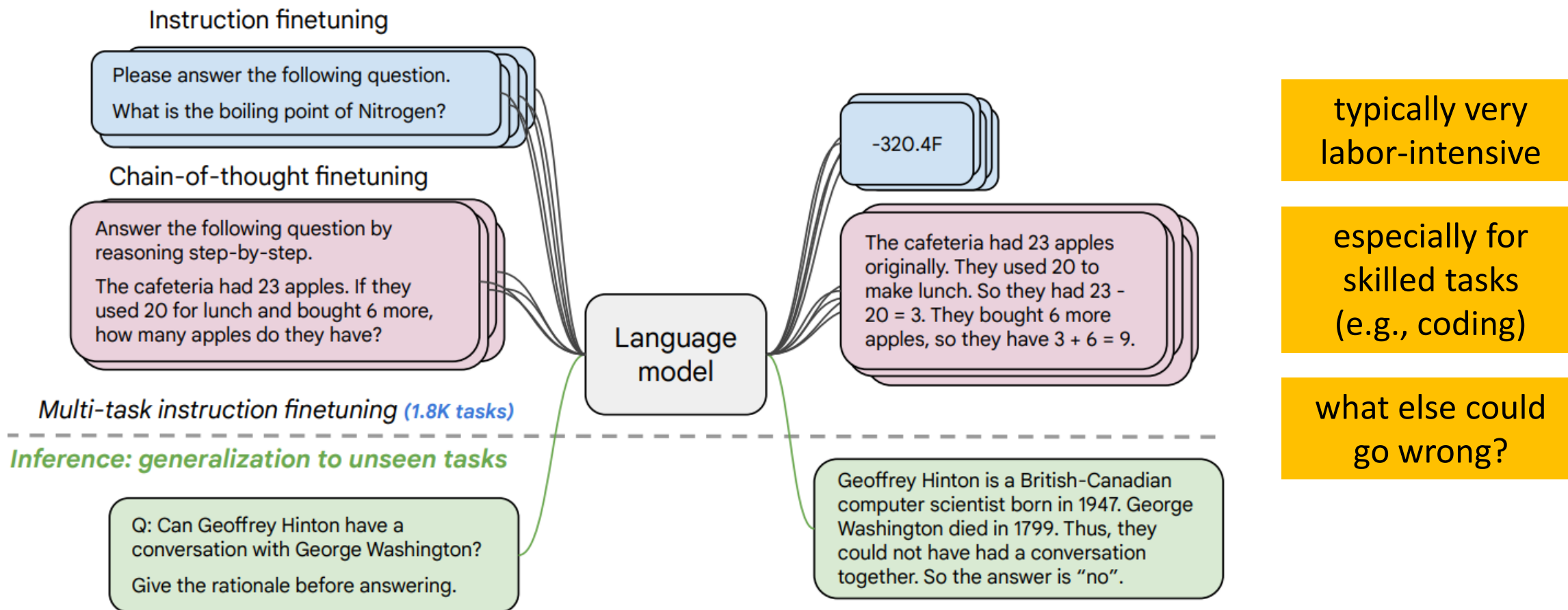
sometimes even fine-tuning LLMs on *incorrect* (e.g., flipped) answers actually gets them to answer questions *correctly* (but in the format shown in fine-tuning)

Unforgettable Generalization in Language Models.

Eric Zhang, Leshem Chosen, Jacob Andreas

LLM post-training

Basic post-training: *instruction tuning*

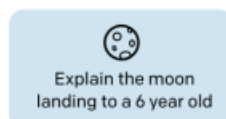


Post-training with RL from preferences

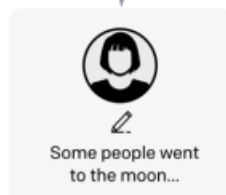
Step 1

Collect demonstration data, and train a supervised policy.

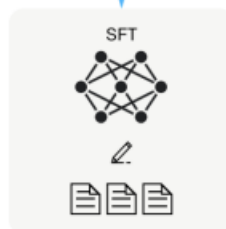
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



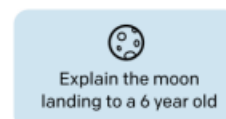
This data is used to fine-tune GPT-3 with supervised learning.



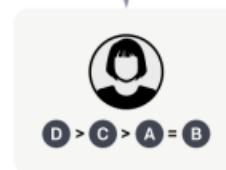
Step 2

Collect comparison data, and train a reward model.

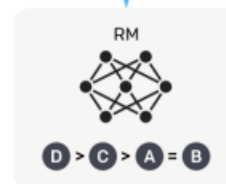
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.

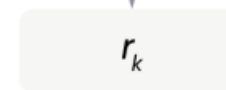


Once upon a time...

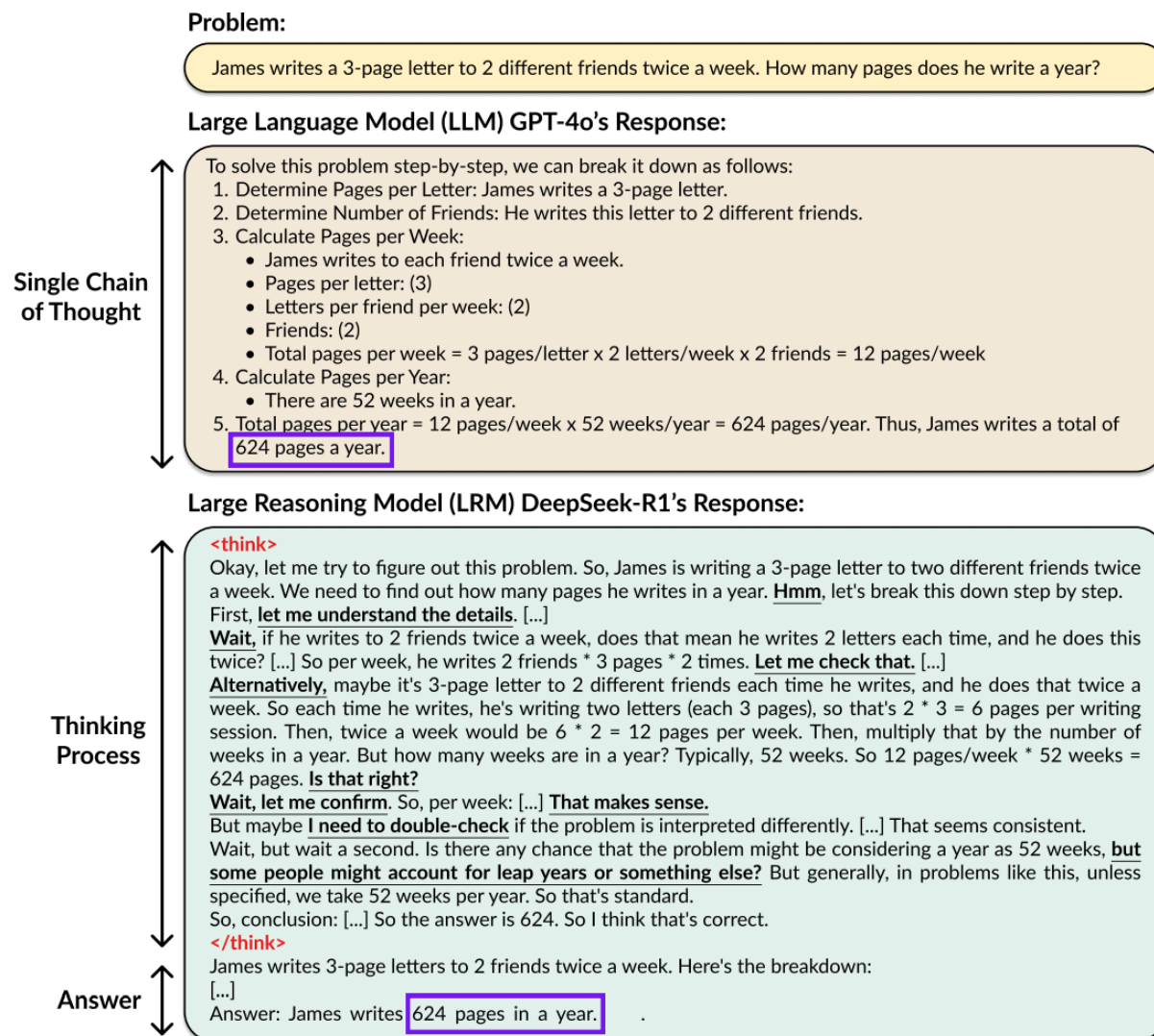
The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Post-training with RL from verifiers

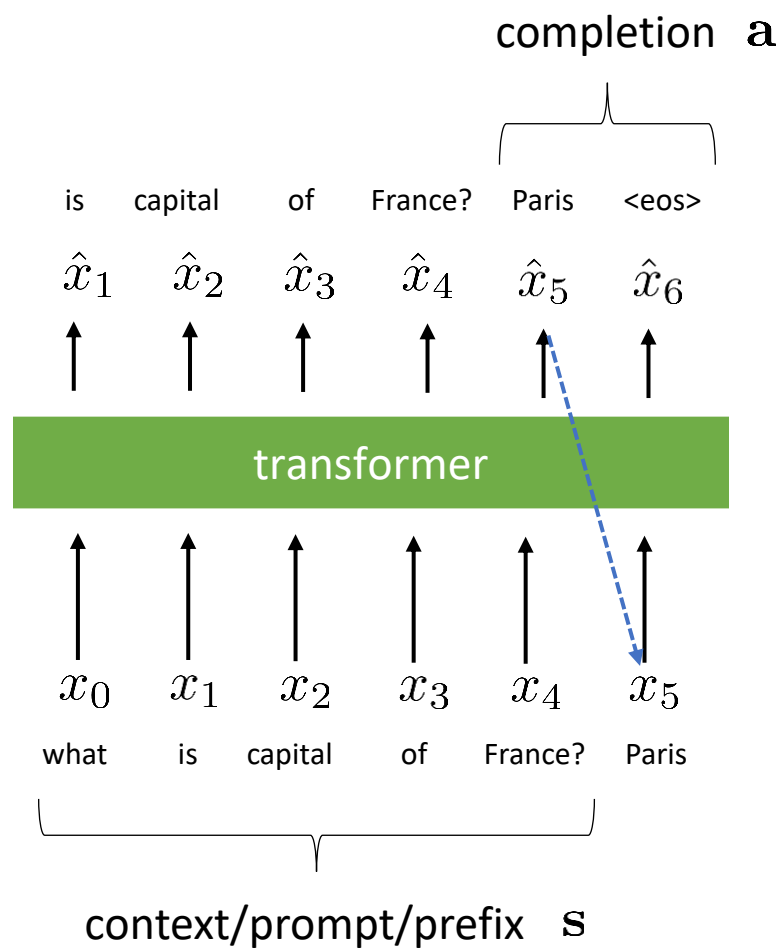


use rewards indicative of answer *correctness*

train model to generate “thoughts” that lead to more correct answers

Figure from Marjanović et al. '25

A basic formulation



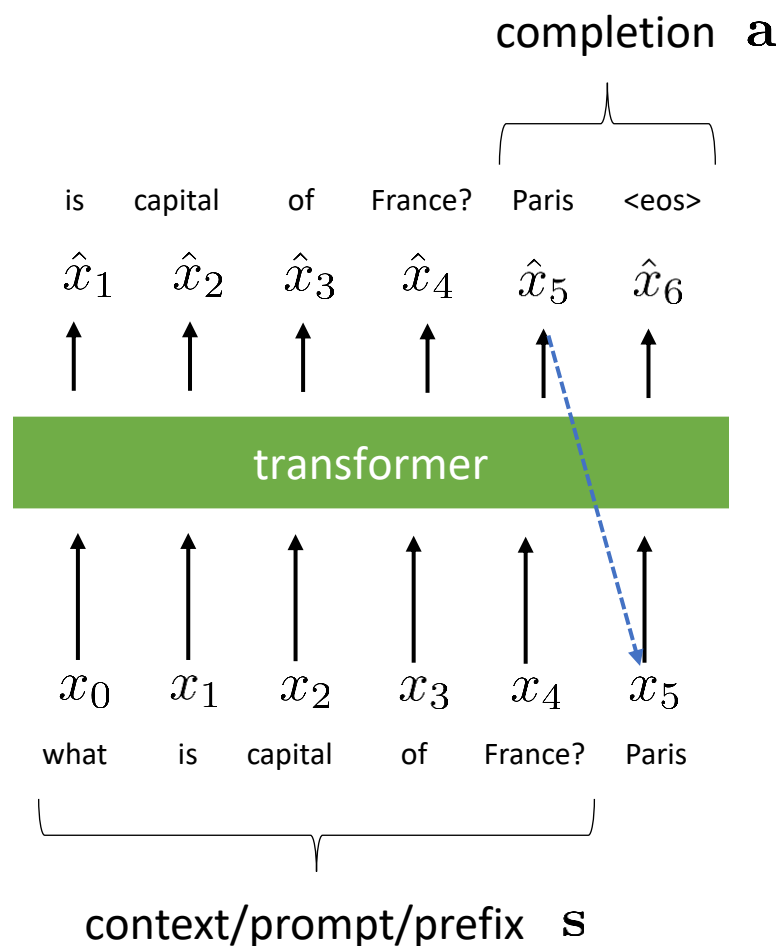
$$\pi_{\theta}(\mathbf{a}|\mathbf{s})$$

$$p(\mathbf{a}|\mathbf{s}) = \underbrace{p(x_5|x_{1:4})}_{\text{prompt}} \underbrace{p(x_6|x_{1:4}, x_5)}_{\text{prompt}}$$

$$E_{\pi_{\theta}(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})]$$

Basic one step RL problem

An equivalent formulation



$$\pi_{\theta}(\mathbf{a}_1|\mathbf{s}_1) \quad \pi_{\theta}(\mathbf{a}_2|\mathbf{s}_2)$$

↓ ↓

$$p(\mathbf{a}|\mathbf{s}) = p(x_5|x_{1:4})p(x_6|x_{1:4}, x_5)$$

prompt prompt

$$E_{\pi_{\theta}(\tau)}[r(\tau)]$$

this perspective makes more sense when we use intermediate rewards and value function baselines

Language models and policy gradients

$$\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) = \nabla_{\theta} \log p(x_5|x_{1:4}) + \nabla_{\theta} \log p(x_6|x_{1:4}, x_5)$$

$$\nabla_{\theta} E_{\pi_{\theta}(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] = E_{\pi_{\theta}(\mathbf{a}|\mathbf{s})}[\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) r(\mathbf{s}, \mathbf{a})]$$

samples from $\pi_{\theta}(\mathbf{a}|\mathbf{s})$

REINFORCE-style
estimator

$$\approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i) r(\mathbf{s}_i, \mathbf{a}_i)$$

importance-weighted
estimator (e.g., PPO)

$$\approx \frac{1}{N} \sum_i \frac{\pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_i|\mathbf{s}_i)} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i) r(\mathbf{s}_i, \mathbf{a}_i)$$

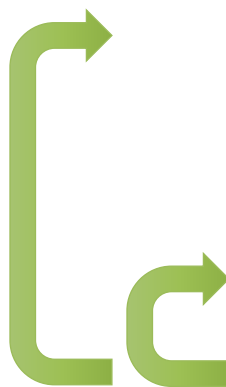
samples from $\bar{\pi}(\mathbf{a}|\mathbf{s})$

Why might we prefer this?

Language models and policy gradients

$$\nabla_{\theta} E_{\pi_{\theta}(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] \approx \frac{1}{N} \sum_i \underbrace{\frac{\pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_i|\mathbf{s}_i)} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i) r(\mathbf{s}_i, \mathbf{a}_i)}_{\hat{\nabla}(\theta, \bar{\pi}, \{\mathbf{a}_i\})}$$

importance-weighted
estimator (e.g., PPO)

- 
1. sample batch $\mathcal{B} = \{\mathbf{a}_i\}$, $\mathbf{a}_i \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$
 2. evaluate $r(\mathbf{s}, \mathbf{a}_i)$ for each $\mathbf{a}_i \in \mathcal{B}$
 3. $\bar{\pi} \leftarrow \pi_{\theta}$
 4. sample *minibatch* $\mathcal{M} \subset \mathcal{B}$
 5. $\theta \leftarrow \theta + \alpha \hat{\nabla}(\theta, \bar{\pi}, \mathcal{M})$
- repeat K times
- what is this?

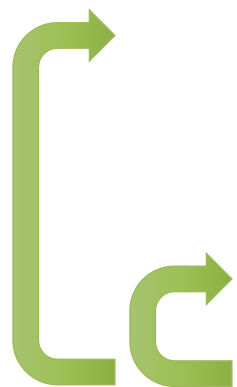
Intermission

Part 3:

Algorithms for RL with LLMs

Language models and policy gradients

$$\nabla_{\theta} E_{\pi_{\theta}(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] \approx \underbrace{\frac{1}{N} \sum_i \frac{\pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_i|\mathbf{s}_i)} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i) r(\mathbf{s}_i, \mathbf{a}_i)}_{\hat{V}(\theta, \bar{\pi}, \{\mathbf{a}_i\})}$$



repeat K times

1. sample batch $\mathcal{B} = \{\mathbf{a}_i\}$, $\mathbf{a}_i \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$
2. evaluate $r(\mathbf{s}, \mathbf{a}_i)$ for each $\mathbf{a}_i \in \mathcal{B}$
3. $\bar{\pi} \leftarrow \pi_{\theta}$
4. sample *minibatch* $\mathcal{M} \subset \mathcal{B}$
5. $\theta \leftarrow \theta + \alpha \hat{V}(\theta, \bar{\pi}, \mathcal{M})$

smartly choose a *baseline* (1)
and a *regularizer* (2)

smartly *set* or *learn* the reward (3)

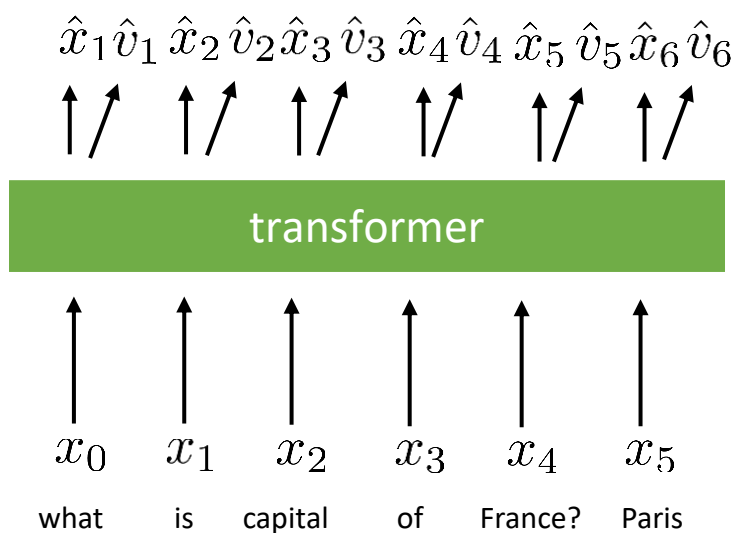
Value function baselines & GAE

most often used with GAE:

$$\delta_{t'} = r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t'+1}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t'})$$

$$\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{\infty} (\gamma \lambda)^{t'-t} \delta_{t'}$$

$$\begin{aligned} \mathcal{L}(\phi) &= \frac{1}{2} \sum_{i=1}^N \left\| \hat{V}_{\phi}^{\pi}(\mathbf{s}_i) - y_i \right\|^2 \\ &= \frac{1}{2} \sum_{i=1}^N \left\| \hat{V}_{\phi}^{\pi}(\mathbf{s}_i) - (\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_{\text{old}}(\mathbf{s}_i)) \right\|^2 \end{aligned}$$



either add a new head or make a copy of the entire network

don't train the value function on the prompt, only the suffix

Baselining without value functions

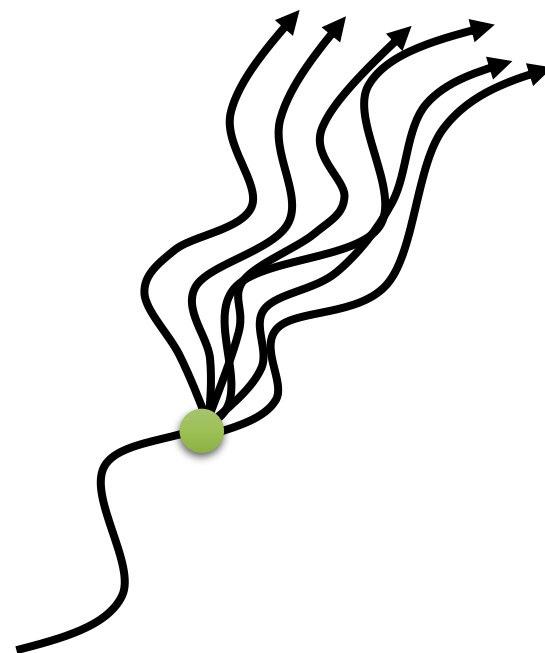
for each prompt $\mathbf{s}_i$

sample $\mathbf{a}_{i,1}, \dots, \mathbf{a}_{i,K}$ from $\pi_\theta(\mathbf{a}|\mathbf{s}_i)$

set $\hat{A}_{i,j}^\pi = r(\mathbf{s}_i, \mathbf{a}_j) - \frac{1}{K} \sum_{k=1}^K r(\mathbf{s}_i, \mathbf{a}_k)$

$$\nabla_\theta E_{\pi_\theta(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] \approx \frac{1}{N} \sum_{ij} \frac{\pi_\theta(\mathbf{a}_j|\mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_j|\mathbf{s}_i)} \nabla_\theta \log \pi_\theta(\mathbf{a}_j|\mathbf{s}_i) \hat{A}_{i,j}^\pi$$

this is called GRPO



Reference model regularization

often we don't want $\pi_\theta(\mathbf{a}|\mathbf{s})$ to drift from some reference model $\pi_{\text{ref}}(\mathbf{a}|\mathbf{s})$

add policy constraint term: $-E_{\pi_\theta}[D_{\text{KL}}(\pi_\theta(\mathbf{a}_t|\mathbf{s}_t), \pi_{\text{ref}}(\mathbf{a}_t|\mathbf{s}_t))]$

$$\bar{r}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) - \beta D_{\text{KL}}(\pi_\theta(\mathbf{a}_t|\mathbf{s}_t), \pi_{\text{ref}}(\mathbf{a}_t|\mathbf{s}_t))$$

avoid producing “nonsense text”
(must be valid English)

avoids “exploiting” imperfect
reward models

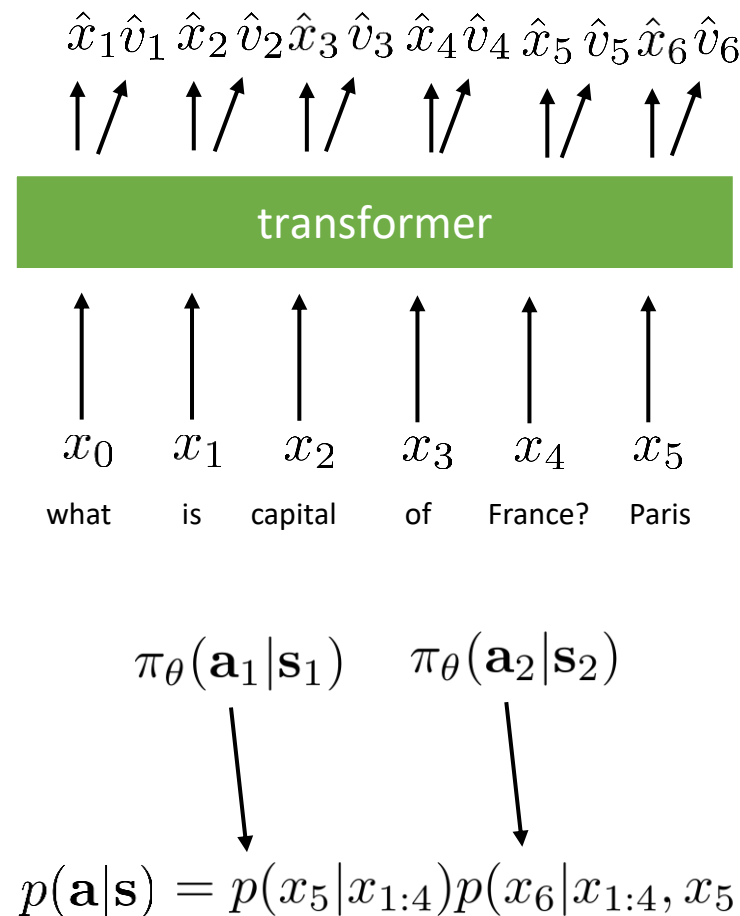
Putting it all together

PPO for LLMs with GAE:

1. sample $\{\tau^{(i)}\}$ from π_θ for each prompt
2. evaluate $\delta_{t,i} = \bar{r}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1,i}) - \hat{V}_\phi^\pi(\mathbf{s}_{t,i})$
3. evaluate $\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_{t,i}, \mathbf{a}_t(i)) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t',i}$
4. fit $\hat{V}_\phi^\pi(\mathbf{s}_i)$ to targets $\{\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_\phi^\pi(\mathbf{s}_i)\}$
5. $\theta' \leftarrow \theta$
- $K \times$
 6. $\nabla_{\theta'} J(\theta') \approx \nabla_{\theta'} \mathcal{L}_{\text{CLIP}}(\theta')$
 7. $\theta' \leftarrow \theta' + \alpha \nabla_{\theta'} J(\theta')$
6. $\theta \leftarrow \theta'$

$$\bar{r}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) - \beta D_{\text{KL}}(\pi_\theta(\mathbf{a}_t | \mathbf{s}_t), \pi_{\text{ref}}(\mathbf{a}_t | \mathbf{s}_t))$$

$$\mathcal{L}_{\text{CLIP}}(\theta') = \sum_{i=1}^N \sum_{t=1}^H \min \left\{ \text{clip} \left(\frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{(i)}, \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} \hat{A}_t^{(i)} \right\}$$



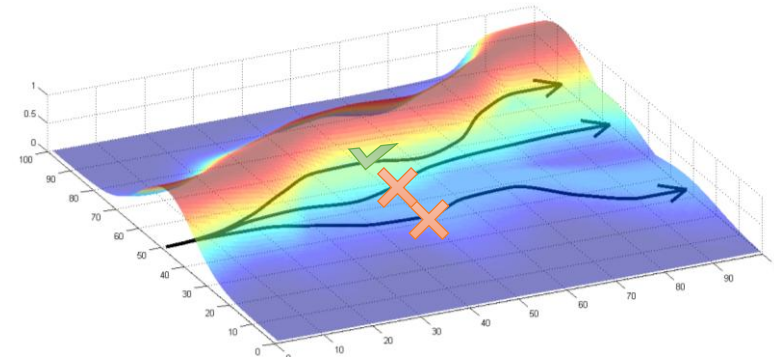
Part 4:

Preferences and verifiers

What if we don't know the reward?

Optometrist algorithm (Bradley-Terry):

- 1. sample 2 (or more) trajectories $\tau_i \sim \pi_\theta(\tau)$
- 2. ask a person to choose preference(s) in $\{\tau_i\}_{i=1}^N$
- 3. train reward r_ψ using preferences
- 4. train policy π_θ with reward r_ψ



preference $\tau_i \succ \tau_j$ means “I like i more than j ”

Bradley-Terry model

How do we train r_ψ ?

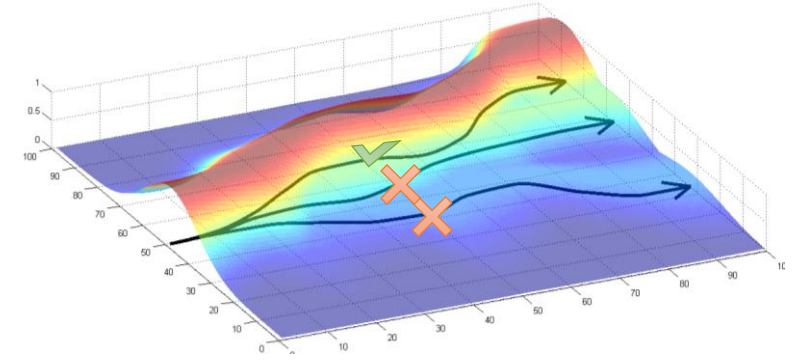
need probabilistic model!

$$p(\tau_i \succ \tau_j) = \frac{\exp\left(\sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})\right)}{\exp\left(\sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})\right) + \exp\left(\sum_t r_\psi(\mathbf{s}_t^{(j)}, \mathbf{a}_t^{(j)})\right)}$$

$$\log p(\tau_i \succ \tau_j) = \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$$

↑
sigmoid function

$$\psi \leftarrow \arg \max_{\psi} \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$$



“better trajectories are exponentially more likely to be chosen based on their reward”

the same method as Elo scores in chess

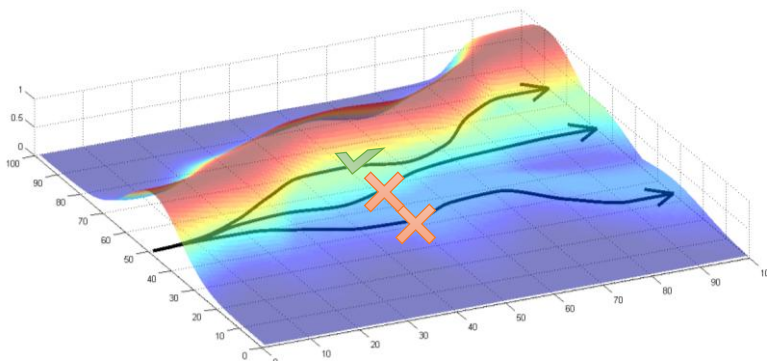
Full RL from human feedback algorithm

RL from human feedback:

1. sample 2 (or more) trajectories $\tau_i \sim \pi_\theta(\tau)$
2. ask a person to specify preferences $\{\tau_i \succ \tau_j\}_{i,j}$
3. train reward $\psi \leftarrow \arg \max_\psi \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$
4. train policy π_θ with reward r_ψ

this looks a lot like logistic regression

this can be incomplete optimization



Often used for post-training with LLMs

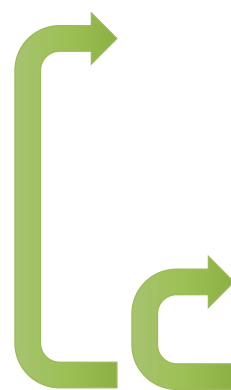
Typically use only 1 or a few iterations

Why?

preference $\tau_i \succ \tau_j$ means “I like i more than j ”

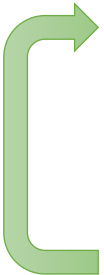
Language models and policy gradients

$$\nabla_{\theta} E_{\pi_{\theta}(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] \approx \underbrace{\frac{1}{N} \sum_i \frac{\pi_{\theta}(\mathbf{a}_i|\mathbf{s})}{\bar{\pi}(\mathbf{a}_i|\mathbf{s})} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}) r(\mathbf{s}, \mathbf{a}_i)}_{\hat{V}(\theta, \bar{\pi}, \{\mathbf{a}_i\})}$$

- 
1. sample batch $\mathcal{B} = \{\mathbf{a}_i\}$, $\mathbf{a}_i \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$
 2. evaluate $r(\mathbf{s}, \mathbf{a}_i)$ for each $\mathbf{a}_i \in \mathcal{B}$
 3. $\bar{\pi} \leftarrow \pi_{\theta}$
 4. sample *minibatch* $\mathcal{M} \subset \mathcal{B}$
 5. $\theta \leftarrow \theta + \alpha \hat{V}(\theta, \bar{\pi}, \mathcal{M})$
- repeat K times
- smartly choose a *baseline* (1)
and a *regularizer* (2)
- smartly set or *learn* the reward (3)

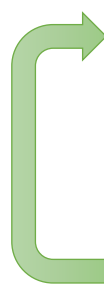
Rewards from human preferences

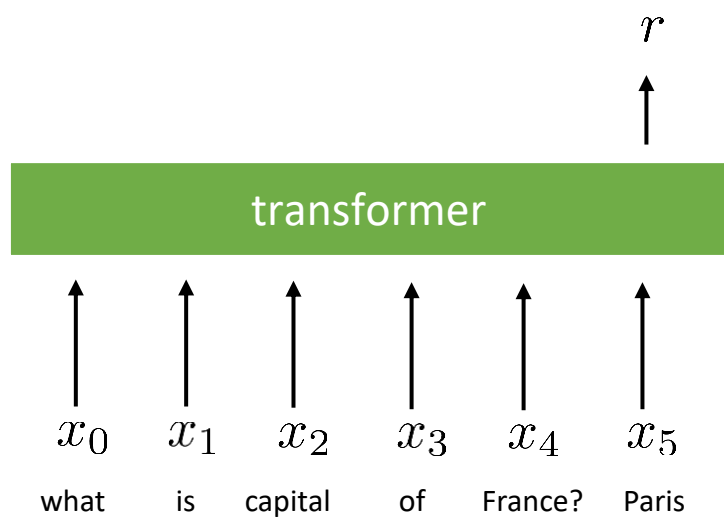
RL from human feedback:

- 
1. sample 2 (or more) trajectories $\tau_i \sim \pi_\theta(\tau)$
 2. ask a person to specify preferences $\{\tau_i \succ \tau_j\}_{i,j}$
 3. train reward $\psi \leftarrow \arg \max_\psi \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$
 4. train policy π_θ with reward r_ψ

RLHF for LLMs

RL from human feedback:

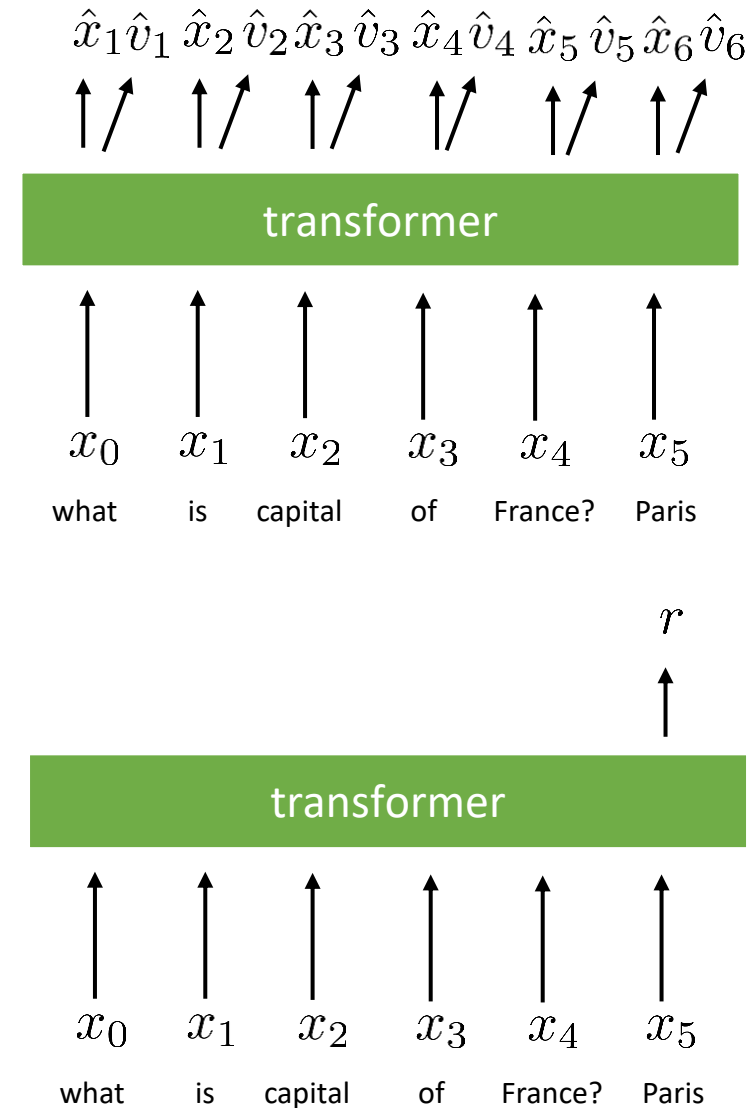
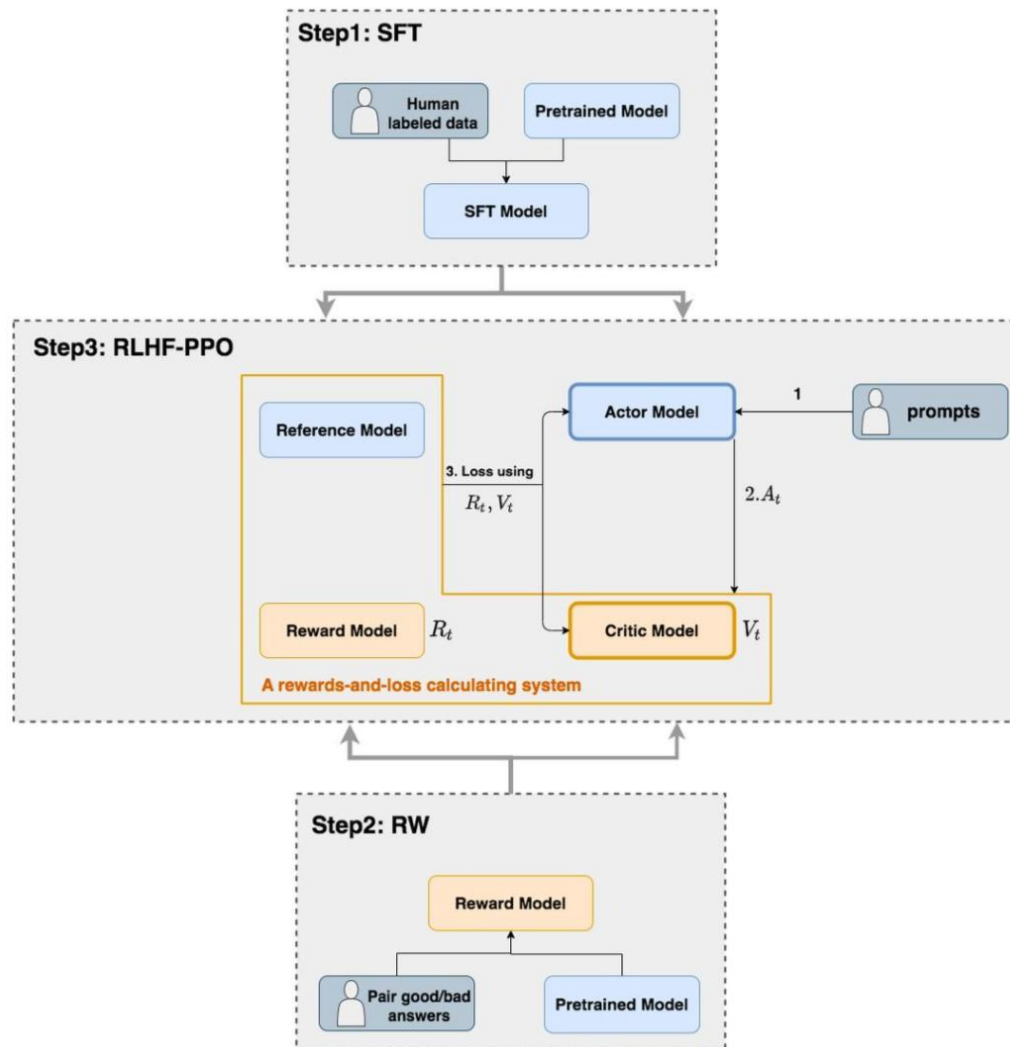
- 
1. sample 2 (or more) trajectories $\tau_i \sim \pi_\theta(\tau)$
 2. ask a person to specify preferences $\{\tau_i \succ \tau_j\}_{i,j}$
 3. train reward $\psi \leftarrow \arg \max_\psi \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$
 4. train policy π_θ with reward r_ψ



Collect preferences over multiple *on policy* responses for the *same* prompt

Reward is assigned only at the *last step* of the generation

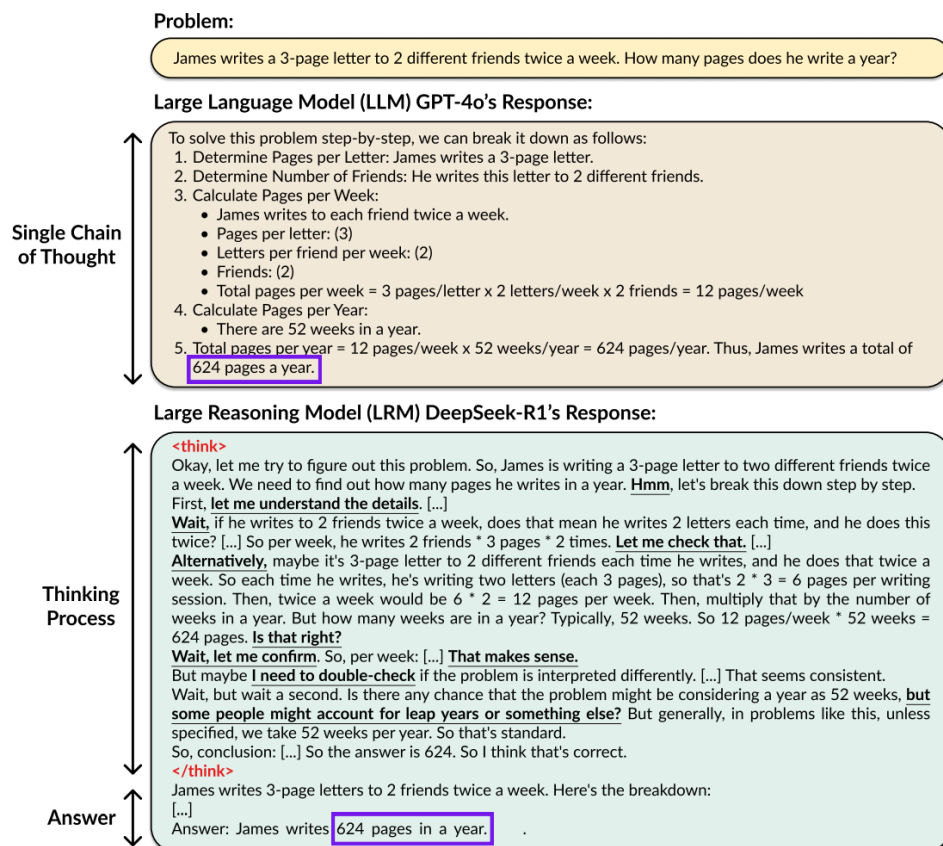
Summary of all the models



Other reward sources

Verifier reward: model evaluates the **answer** and indicates if it is correct or not

Process reward: model evaluates each step of chain-of-thought generation, indicating if it is correct or not



Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

This problem is about factoring a polynomial over the integers, which can be a tricky task.

One possible strategy is to look for rational roots of the polynomial, using the rational root theorem.

However, that would involve checking a lot of possible factors of -4 , and there is no guarantee that the polynomial has any rational roots at all.

Another possible strategy is to use some algebraic tricks to simplify the polynomial or reduce its degree.

I notice that the polynomial has only even powers of x , so I can try to substitute $y = x^2$ and see what happens.

Then the polynomial becomes $y^4 + 3y^2 - 4$, which is a quadratic equation in y^2 .

I can try to factor this quadratic equation by looking for two numbers that multiply to -4 and add to 3 .

Those numbers are 4 and -1 , so I can write $y^4 + 3y^2 - 4 = (y^2 + 4)(y^2 - 1)$.

Now I can substitute back $x^2 = y$ and get $x^8 + 3x^4 - 4 = (x^4 + 4)(x^4 - 1)$.

This looks promising, because I have reduced the degree of the factors from 8 to 4 .

I can try to factor each of these factors further, using the difference of squares formula.

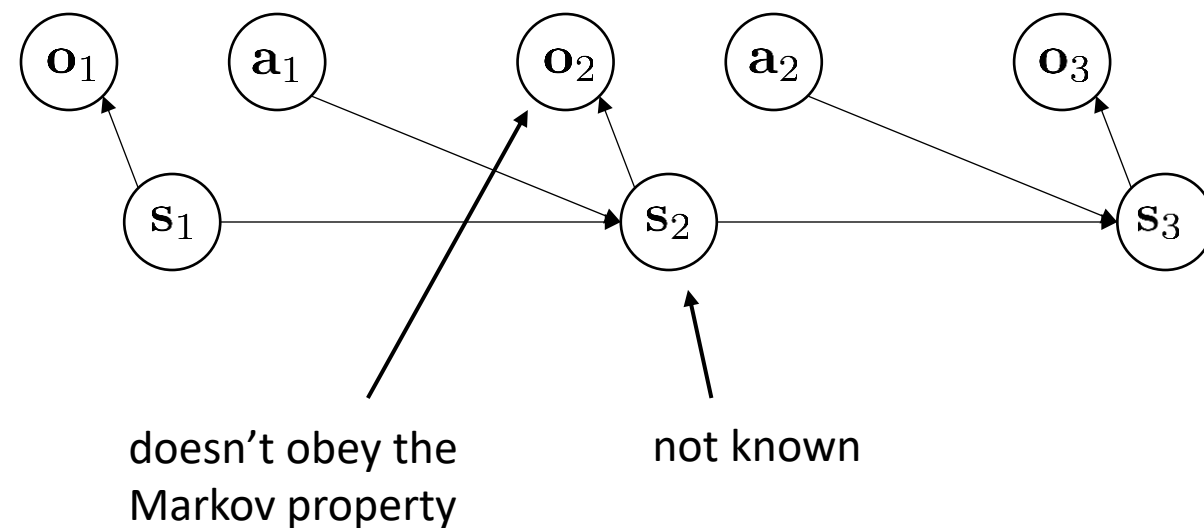
I get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x^2 - 1)$.

I can apply the difference of squares formula again to the last factor and get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x + 1)(x - 1)$.

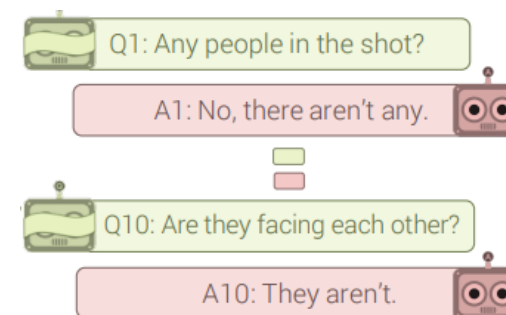
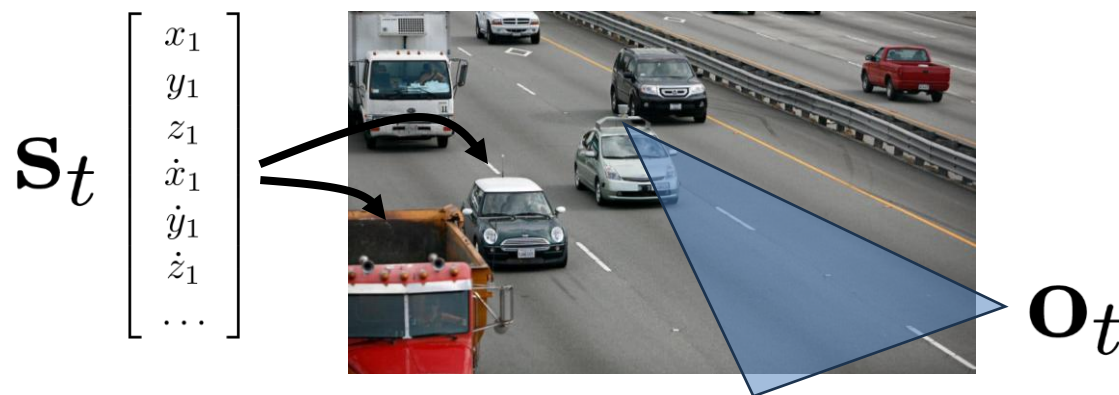
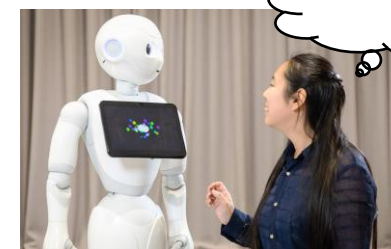
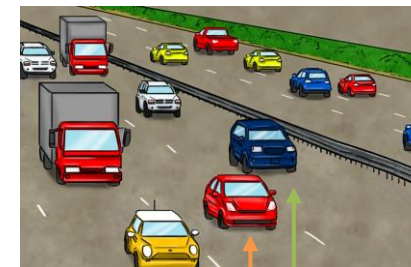
Now I have factored the polynomial completely into monic linear and quadratic factors with integer coefficients.

Part 5 (bonus):
RL with partial observability

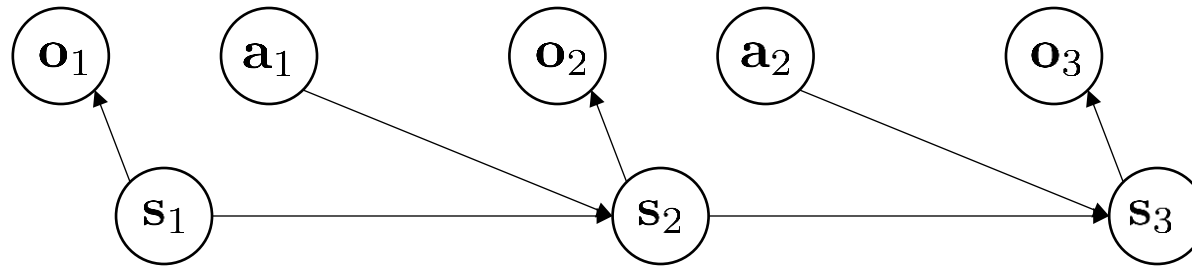
Beyond MDPs



most real-world problems are like this!



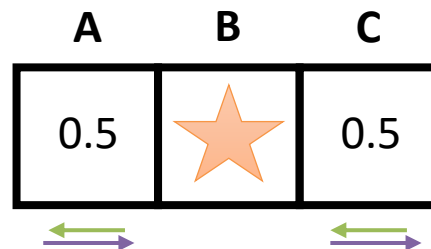
Partially observed MDPs can be weird



Example 1: information-gathering actions



Example 2: stochastic optimal policies



Which methods handle partial observability?

Policy gradients

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \xrightarrow{??} \nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{o}_{i,t}) A(\mathbf{o}_{i,t}, \mathbf{a}_{i,t})$$

Value-based methods

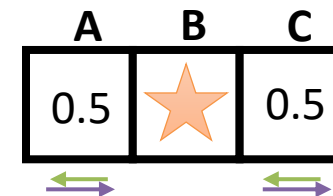
$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{s}', \mathbf{a}') \xrightarrow{??} Q(\mathbf{o}, \mathbf{a}) \leftarrow r(\mathbf{o}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{o}', \mathbf{a}')$$

Trick question:
what does “handle” mean?

$$\pi_{\theta}(\mathbf{a} | \mathbf{o})$$

Model-based RL methods

$$\hat{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \xrightarrow{??} \hat{p}(\mathbf{o}' | \mathbf{o}, \mathbf{a}) \quad \text{more on this later!}$$



handle = find best policy in policy class

Which methods handle partial observability?

Policy gradients

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \xrightarrow{??} \nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{o}_{i,t}) A(\mathbf{o}_{i,t}, \mathbf{a}_{i,t})$$

this is OK (no Markov property!)

Key point: advantage is a function of the state $\mathbf{s}_t$

it does *not* depend on $\mathbf{s}_{t-1}$

that's why it's OK to use $r_t + \gamma \hat{V}(\mathbf{s}_{t+1}) - \hat{V}(\mathbf{s}_t)$

it is *not* OK to train $\hat{V}(\mathbf{o}_t)$

this takes some care

every time we see this state,
we expect to get this value,
regardless of past states

past observations **do**
matter for this value

Which methods handle partial observability?

Policy gradients

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \xrightarrow{??} \nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{o}_{i,t}) A(\mathbf{o}_{i,t}, \mathbf{a}_{i,t})$$

this is OK (no Markov property!)

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{o}_{i,t}) \right) \left(\sum_{t=1}^H r(\mathbf{o}_{i,t}, \mathbf{a}_{i,t}) \right) \quad \checkmark$$

this takes some care

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{o}_{i,t}) (r_{i,t} + \gamma \hat{V}(\mathbf{o}_{i,t+1}) - \hat{V}(\mathbf{o}_{i,t})) \quad \times$$

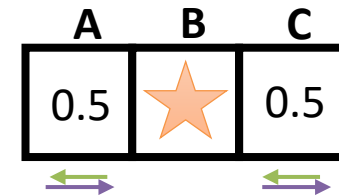
Pop quiz:
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{o}_{i,t}) \left(\sum_{t'=t}^H r(\mathbf{o}_{i,t'}, \mathbf{a}_{i,t'}) \right)$$

Which methods handle partial observability?

Value-based methods

$$Q(s, a) \leftarrow r(s, a) + \gamma \max_{a'} Q(s', a') \quad \xrightarrow{??} \quad Q(o, a) \leftarrow r(o, a) + \gamma \max_{a'} Q(o', a')$$

Value-based methods do not work
without the Markov property



Remember:

Key point: advantage is a function of the state s_t

it does *not* depend on s_{t-1}

that's why it's OK to use $r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$

it is *not* OK to train $\hat{V}(o_t)$

every time we see this state,
we expect to get this value,
regardless of past states

past observations **do**
matter for this value

Which methods handle partial observability?

Model-based RL methods $\hat{p}(s'|s, a) \xrightarrow{??} \hat{p}(o'|o, a)$ ❌

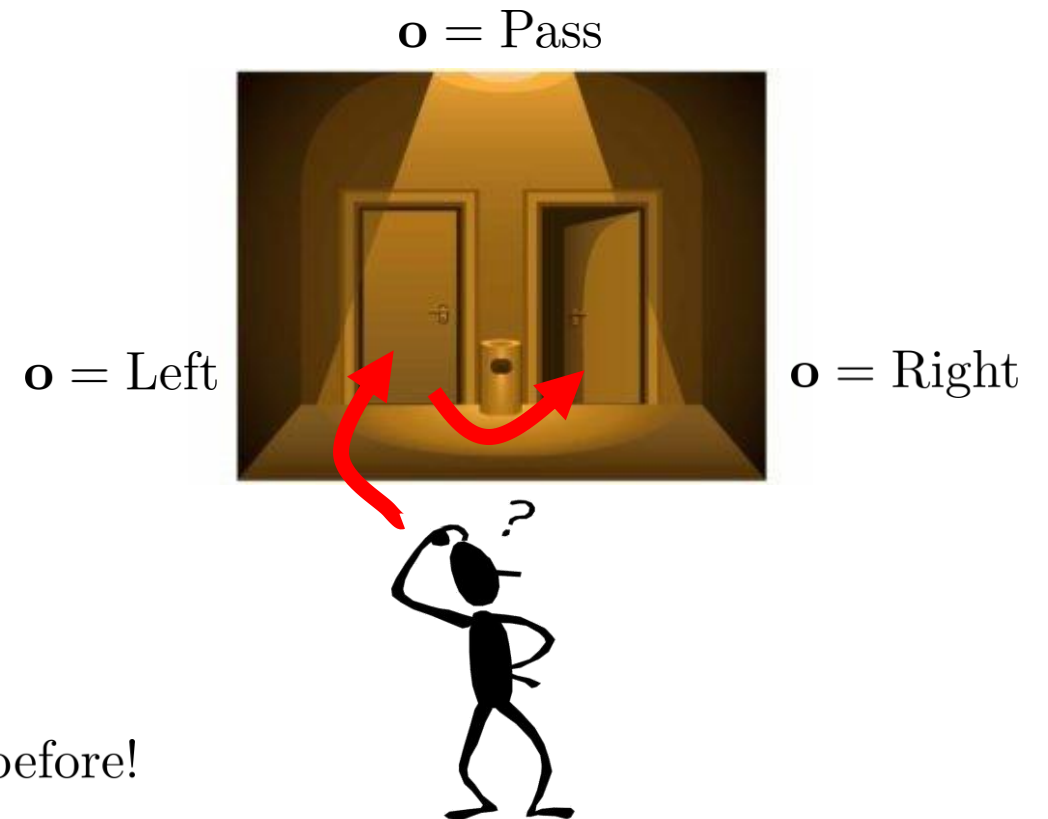
Example showing why this is such a bad idea:

$$p(o' = \text{Pass} | o = \text{Left}, a = \text{Open}) = 0.5$$

50% probability to open *each time* you try

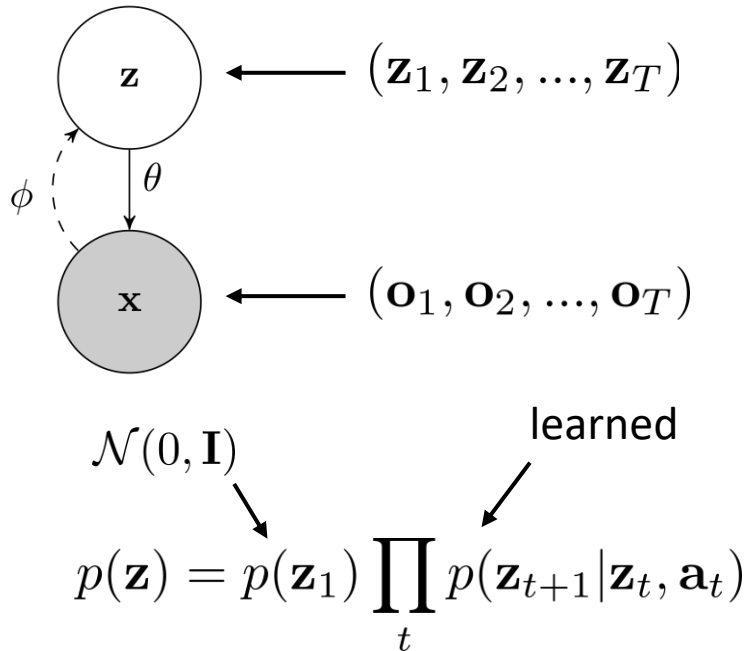
Just keep trying!

$$p(o' = \text{Pass} | o = \text{Left}, a = \text{Open}) = 0 \text{ if it didn't open before!}$$



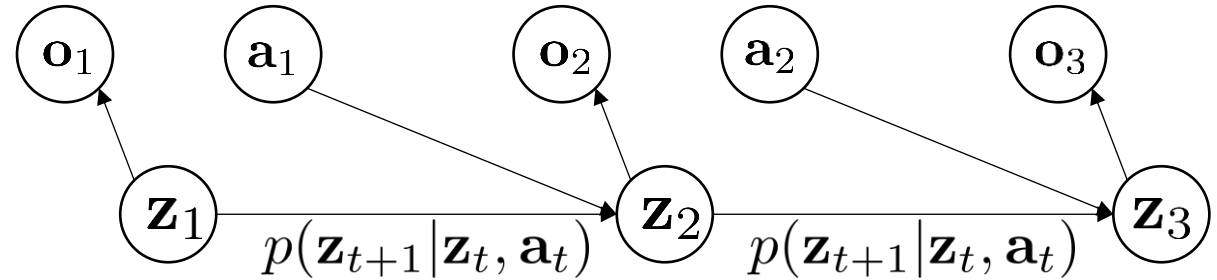
State space models

Can we *learn* a Markovian state space?



$$p_{\theta}(\mathbf{o} | \mathbf{z}) = \prod_t p(\mathbf{o}_t | \mathbf{z}_t)$$

$$q_{\phi}(\mathbf{z} | \mathbf{o}) = \prod_t q_{\phi}(\mathbf{z}_t | \mathbf{o}_{1:t})$$



This can work quite well!

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{s}', \mathbf{a}')$$

$$Q(\mathbf{o}, \mathbf{a}) \leftarrow r(\mathbf{o}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{o}', \mathbf{a}')$$

$$Q(\mathbf{z}, \mathbf{a}) \leftarrow r(\mathbf{z}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{z}', \mathbf{a}')$$

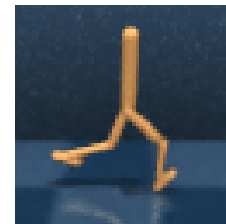


We'll discuss these types of methods more in the model-based RL section

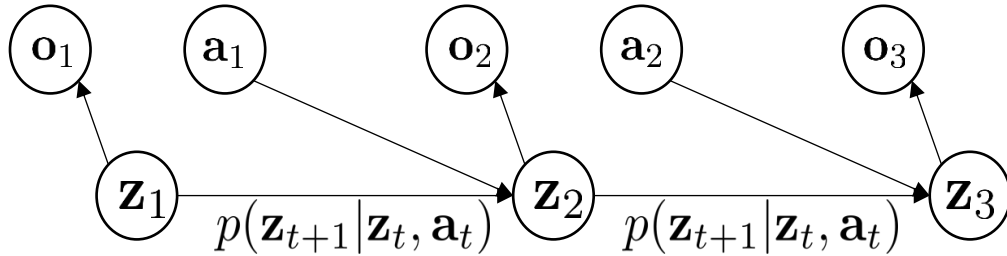
Why might this not be enough?

Prediction can be *hard*

Maybe we don't need good prediction to get high rewards

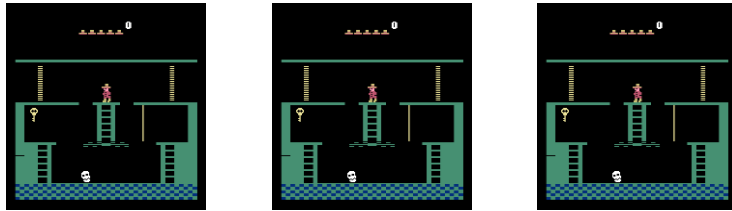


History states



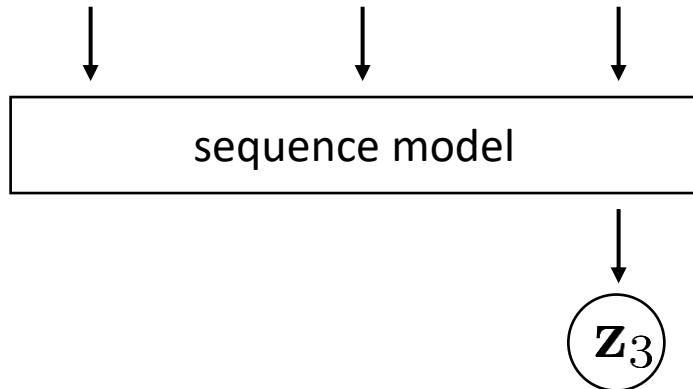
$$q_{\phi}(\mathbf{z}|\mathbf{o}) = \prod_t q_{\phi}(\mathbf{z}_t|\mathbf{o}_{1:t})$$

← state is inferred from a *history*



state is a *function* of history

can we just give the history to our value function?



$$\mathbf{s}_t = (\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_t)$$

Does that work?

Does that obey the Markov property?

tells *nothing* we didn't know from $\mathbf{s}_t$!

$$\mathbf{s}_{t+1} \perp \mathbf{s}_{t-1} | \mathbf{s}_t$$

$(\mathbf{o}_1, \dots, \mathbf{o}_{t-1})$ $(\mathbf{o}_1, \dots, \mathbf{o}_t)$

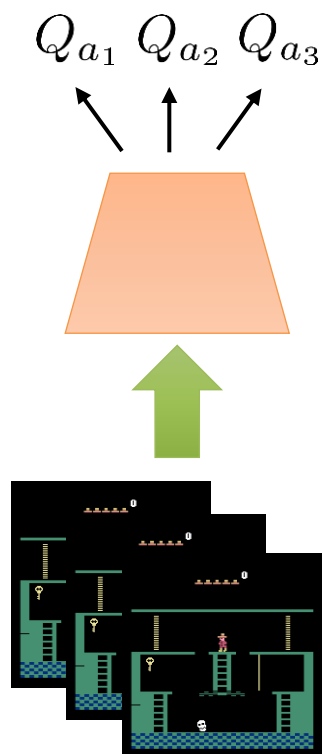
$$Q(\mathbf{o}_1, \dots, \mathbf{o}_t, \mathbf{a}) \leftarrow r(\mathbf{o}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{o}_1, \dots, \mathbf{o}_{t+1}, \mathbf{a}')$$



Model architectures

$$Q(\mathbf{o}_1, \dots, \mathbf{o}_t, \mathbf{a}) \leftarrow r(\mathbf{o}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{o}_1, \dots, \mathbf{o}_{t+1}, \mathbf{a}')$$

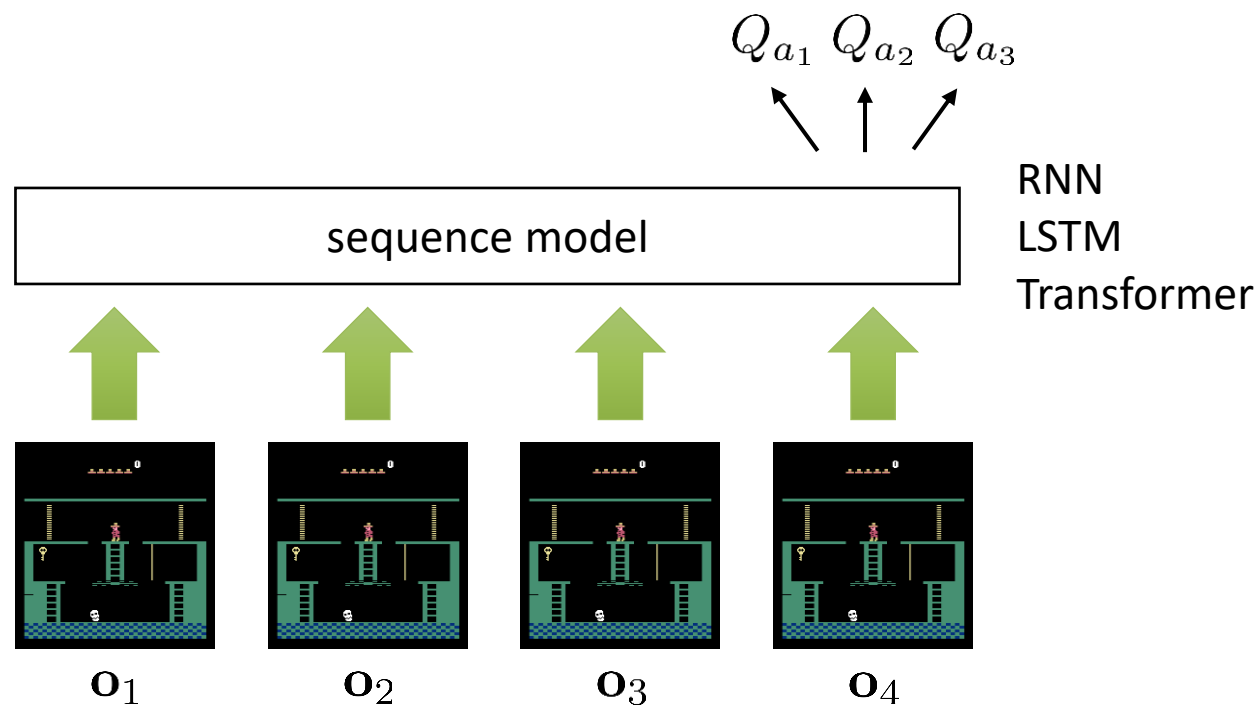
how to represent this?



fixed (short)
history

Is that bad?

Sometimes...



Recap & overview

- POMDPs are *weird*
- Some methods “just do it”
 - But most efficient ones don’t, because they require value functions
 - Even those that do only get the best *memoryless* policy
- We could *learn* a Markovian state space with models
- We could also just use *history states*, which just means using a sequence model to read in observation histories

